# Testing the Jasmin ARMv8-A Back End on the Raspberry Pi 5

Differential, ABI, cryptographic, constant-time and EasyCrypt validation of Jasmin 2026.09.0 and EasyCrypt r2026.07

October 21, 2026

**Abstract**

Jasmin 2026.09.0 is the first release whose verified compiler generates code for ARMv8-A (AArch64), the 64-bit Arm architecture. The upstream test suite of Jasmin verifies only that the ARMv8-A test programs *compile*; it does not execute the generated code. This report describes a test campaign, conducted entirely on a Raspberry Pi 5 (Cortex-A76), that executes the generated code natively and compares its results with several independent references: the reference interpreter of Jasmin, hand-written C models, built-in functions of GCC, the test vectors of RFC 8439, and the OpenSSL library. The campaign also examines the AAPCS64 calling convention, the erasure of secrets from the stack, and constant-time security (with the checker `jasmin-ct`, with timing measurements, and with EasyCrypt proofs on the extracted leakage model), and it proves in EasyCrypt the functional correctness of the carry handling. All **161 checks pass** in 7 minutes. More than 10 million executions of generated code reveal **no miscompilation**, that is, no case in which the compiled code computes a result different from the one that the language semantics prescribes. The report also documents 17 limitations of the back end. Three of them are internal compiler errors (for the operations `mulu` and the 32-bit `swap`). Speculative load hardening, a protection against Spectre attacks, is accepted by the checker but rejected by the compiler. The instructions `UMULL` and `UMADDL` are defined in the EasyCrypt model but not available in the compiler, which makes Poly1305 2.5 times slower than C. Each limitation is accompanied by a workaround that has been tested.

## Contents

**1 Introduction** **3**
- 1.1 Why test a verified compiler? . . . 3
- 1.2 Terms used in this report . . . 3
- 1.3 Objectives and structure . . . 3
- 1.4 Summary of results . . . 4

**2 Platform and test architecture** **5**
- 2.1 Oracles . . . 5
- 2.2 Directory structure and usage . . . 5

**3 Stage 1: upstream tests on the ARMv8-A target** **6**

**4 Stage 2: differential testing** **7**
- 4.1 A reference interpreter for ARMv8-A . . . 7
- 4.2 The differential tester . . . 8
- 4.3 Instruction coverage beyond the upstream tests . . . 9
- 4.4 Independent C models . . . 10

**5 Stage 3: AAPCS64 conformance** **10**

**6 Stage 4: cryptographic kernels** **11**
6.1 ChaCha20 . . . 11
6.2 Poly1305 without a wide multiplier . . . 12
6.3 Results . . . 13

**7 Stage 5: constant time** **13**
7.1 Static verification with `jasmin-ct` . . . 13
7.2 Timing measurements . . . 15

**8 Stage 6: EasyCrypt** **15**
8.1 Extraction . . . 16
8.2 Constant-time proofs . . . 16
8.3 Functional correctness of the carry handling . . . 17

**9 Stage 7: performance** **18**

**10 Stage 8: limitations of the back end** **19**

**11 Conclusion and recommendations** **19**

**A Glossary** **20**

**B Complete output of `run-all.sh`** **20**

# 1 Introduction

The companion document `../install-guide` describes the installation of Jasmin and EasyCrypt on a Raspberry Pi 5 and a set of smoke tests. The present document addresses two further questions: *can the new ARMv8-A back end be trusted, and which programs does it currently support?* The reader is assumed to have read the installation guide or to be familiar with its content; terms that it does not introduce are explained below and in the glossary (Appendix A).

## 1.1 Why test a verified compiler?

The correctness of the Jasmin compiler is proved in Coq: for every program that the compiler accepts, the generated code has the same behavior as the source program. This proof covers the compiler passes, but not every component on which a user relies. It does not cover the OCaml components that are not extracted from Coq (the parser and some auxiliary algorithms written in OCaml), the printing of the assembly text, the assumptions about the semantics of the processor instructions, the platform ABI (the conventions by which separately compiled functions interoperate), or the assembler and linker. Moreover, the proof does not specify which source programs the back end accepts. Testing on real hardware complements the proof in all these respects.

## 1.2 Terms used in this report

**Back end** The part of the compiler that generates code for one processor architecture, here ARMv8-A.

**Miscompilation** A case in which the compiled code computes a result that differs from the result that the language semantics prescribes.

**Oracle** An independent source of expected results against which a test compares the results of the code under test.

**Differential testing** Testing that executes two implementations of the same function on the same inputs and compares their results.

**AAPCS64** The procedure call standard of AArch64, which specifies how functions receive arguments and return results, and which registers they must preserve.

**Constant time, leakage** A program is constant-time if secret data influence neither its control flow nor its memory addresses. The *leakage* of an execution is the sequence of its branch decisions and memory addresses, which determine its timing and which an attacker who measures time may therefore learn.

**Spectre v1, SLH** Spectre v1 is an attack that exploits the speculative execution of conditional branches. Speculative load hardening (SLH) is a programming technique that protects against it.

**ICE** Internal compiler error: the compiler fails with an internal error instead of producing code or a diagnostic for the user.

## 1.3 Objectives and structure

The campaign consists of eight stages, each of which has its own directory and its own section in this report. Each section states the purpose of the stage, describes the method, and presents the results. Problems of the back end are highlighted as *findings*.

1. Compile the upstream ARMv8-A tests and the architecture-independent tests for the ARMv8-A target (Stage 1).

2. Execute the compiled code and compare its results with the language semantics by *differential testing* (Stage 2).
3. Verify the AAPCS64 calling convention and the erasure of secrets from the stack on the target machine (Stage 3).
4. Implement two cryptographic primitives (ChaCha20, Poly1305) and validate them against the standard and against OpenSSL (Stage 4).
5. Validate the constant-time guarantees statically, empirically and formally (Stages 5–6).
6. Measure performance (Stage 7) and catalog the limitations of the back end (Stage 8).

### 1.4 Summary of results

Table 1 lists the checks of each stage. A check is a single pass/fail decision of the test script; the complete output of a run is reproduced in Appendix B.

| Stage | Topic | Content | Checks |
|---|---|---|---|
| 1 | Upstream tests | 72 programs compiled and assembled, 27 test cases of the constant-time checker, 17 translations to EasyCrypt type-checked | 47 |
| 2 | Differential testing | 35 functions × 2000–5000 inputs (interpreter vs native code), $2 \times 10^6$ tests against C models | 37 |
| 3 | ABI | callee-saved registers, 32-bit arguments with undefined upper halves, erasure of secrets from the stack | 17 |
| 4 | Cryptography | ChaCha20 and Poly1305: RFC 8439 test vectors, 262 608 comparisons with OpenSSL | 4 |
| 5 | Constant time | `jasmin-ct` (modes CT, DOIT, SCT), 200 000 functional tests, 4 timing tests | 24 |
| 6 | EasyCrypt | 10 extracted models, 2 correctness proofs and 5 constant-time proofs, 3 negative proofs | 15 |
| 7 | Performance | ChaCha20 and Poly1305 compared with C and OpenSSL | info |
| 8 | Limitations | 17 expected compilation failures | 17 |
| | **Total** | | **161 / 161** |

Table 1: Overview of the test campaign.

**Principal findings.**

- **No miscompilation**: more than $10^7$ executions of Jasmin-generated AArch64 code agree with the reference semantics and with independent implementations.
- The generated code conforms to AAPCS64: it preserves the callee-saved registers and the stack pointer, and it ignores the undefined upper halves of 32-bit arguments. The erasure option of the compiler (stack zeroization) removes every secret word from the stack.
- **Internal compiler errors** occur for `mulu` (64-bit and 32-bit widening multiplication) and for the 32-bit `#swap` (Finding 1).
- The ARMv8-A back end provides **no speculative load hardening**, although `jasmin-ct --sct` accepts programs that use it for this architecture (Finding 3).
- **UMULL and UMADDL** are defined in the EasyCrypt model but not available in the compiler; as a consequence, Poly1305 is 2.5 times slower than C (Finding 4).
- The register allocator fails to allocate the borrow flag of a subtraction with borrow. The workaround relies on the A64 carry convention (C = NOT borrow), whose correctness is *proved* in EasyCrypt (Finding 2).

# 2 Platform and test architecture

| | |
|---|---|
| Board | Raspberry Pi 5 Model B Rev 1.0, 4 × Cortex-A76 at 2.4 GHz, 8 GB RAM |
| Operating system | Debian 12 (bookworm), kernel 6.12.93+rpt-rpi-2712, aarch64, glibc 2.36 |
| Jasmin | 2026.09.0 (opam switch `jasmin-ec`, OCaml 4.14.2) |
| EasyCrypt | r2026.07; provers Alt-Ergo 2.6.0, CVC5 1.0.3, Z3 4.8.12 |
| C and cryptography | GCC 12.2.0, GNU as 2.40, OpenSSL 3.0.20 (`libssl-dev`) |

Table 2: Test platform (installation procedure: `../install-guide`).

## 2.1 Oracles

A test can detect an error only if the correct result is known; the quality of a test therefore depends on its oracle. Each result of an execution is compared with at least one reference that is independent of the code under test (Figure 1):

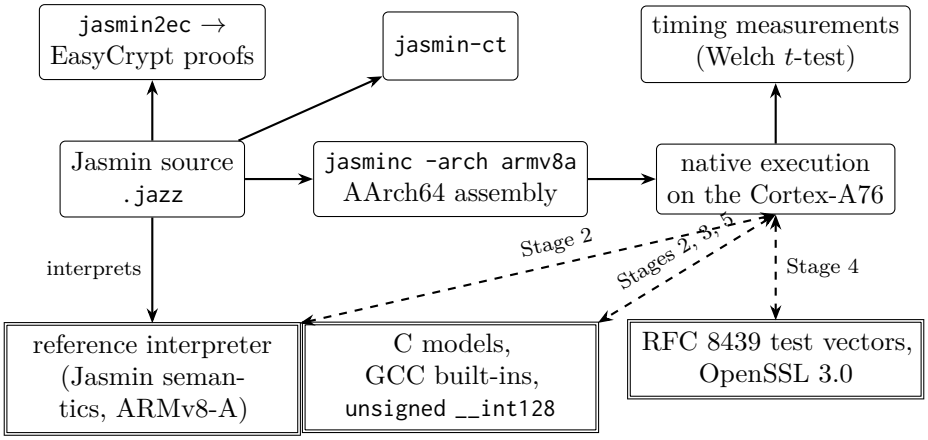



Figure 1: Tools under test (single frame) and oracles (double frame). Solid arrows show how the tools process the source program; dashed arrows denote comparisons of results.

- The **reference interpreter** executes the *source* program directly, with the ARMv8-A instruction semantics of the Jasmin library, that is, with the semantics to which the compiler correctness theorem refers. Agreement between the interpreter and the native execution therefore tests the entire chain: compiler, assembly printer, assembler, linker, ABI and processor.
- **Independent implementations** (C models, OpenSSL, RFC test vectors) additionally detect errors in the *algorithms* of the test programs. The interpreter and the compiler process the same source program, so their agreement alone cannot detect such errors.

## 2.2 Directory structure and usage

```
armv8-tests/
|-- run-all.sh               # runs the 8 stages, prints [PASS]/[FAIL]/[INFO]
|-- results/run-all-output.txt
|-- lib/testlib.h            # PRNG (splitmix64), edge values, TL_CHECK
|-- tools/
|   |-- jasmin-interp/       # OCaml reference interpreter driver (dune)
|   |-- difftest.py          # interpreter-vs-native differential tester
|   `-- negtest.sh           # expected-failure runner
|-- 01-upstream/             # upstream tests (Jasmin v2026.09.0, MIT)
|-- 02-isa/                  # isa.jazz, test_models.c, test_ldst.c
|-- 03-abi/                  # trampoline.S, stackzero.jazz, test_abi.c
|-- 04-crypto/               # chacha20.jazz, poly1305.jazz + tests
```

```
|-- 05-ct/                  # ct.jazz, sct.jazz, test_ct.c
|-- 06-easycrypt/           # extracted/ (generated), proofs/
|-- 07-bench/               # bench.c
|-- 08-negative/            # 17 programs the back end must reject
`-- report/                 # this document
```

The script `run-all.sh` runs all stages, or only the stages given as arguments. It prints one line per check, marked `[PASS]`, `[FAIL]` or `[INFO]` (for measurements without a pass/fail criterion), and a summary at the end:

```
$ eval $(opam env --switch=jasmin-ec)
$ cd armv8-tests
$ ./run-all.sh                # all stages (about 7 minutes)
$ ./run-all.sh 2 4            # only stages 2 and 4
```

Apart from the installation described in the installation guide, the tests require only the Debian package `libssl-dev` (the development files of OpenSSL). All random inputs originate from a pseudo-random generator with a fixed seed, so that every run is reproducible.

## 3 Stage 1: upstream tests on the ARMv8-A target

*Purpose: determine which of the test programs of the Jasmin developers the ARMv8-A back end compiles, and whether the generated code is valid ARMv8-A assembly.*

Three groups of tests were copied from the Jasmin v2026.09.0 sources: the 16 ARMv8-A-specific tests (`tests/success/armv8a`), the ARMv8-A stack-zeroization test, and the 55 architecture-independent tests (`tests/success/common`). The upstream test suite runs the *common* tests only for x86-64, ARM-M4 and RISC-V. Each program is compiled for ARMv8-A and then assembled with the GNU assembler `as` (invoked through `gcc -c`), which verifies that every emitted instruction can be encoded as machine code:

```
$ jasminc -arch armv8a -o build/up_add.s 01-upstream/armv8a/add.jazz
$ gcc -c -o build/up_add.o build/up_add.s
```

The stage also runs the upstream tests of the constant-time checker (in the directories `CCT/success` and `CCT/fail`, which contain programs that the checker must accept and programs that it must reject) for the ARMv8-A architecture, and it translates the ARMv8-A programs to EasyCrypt and type-checks the results. Table 3 summarizes the outcome.

| Group | Programs | Passed | Remarks |
|---|---|---|---|
| `armv8a` | 16 | 16 | |
| `stack_zeroization/armv8a` | 1 | 1 | |
| `common` (not run upstream for ARMv8-A) | 55 | 51 | 4 internal compiler errors |
| `CCT/success` with `jasmin-ct --arch=armv8a` | 15 | 15 | accepted |
| `CCT/fail` with `jasmin-ct --arch=armv8a` | 12 | 12 | rejected |
| extraction and EasyCrypt type check (`armv8a`, stack zeroization) | 17 | 17 | 6–10 s each |

Table 3: Upstream tests on the ARMv8-A target.

> **Finding 1: internal compiler errors.** Four common tests (`bug_499`, `renaming`, `swap`, `swap_word`) cause the ARMv8-A back end to fail with an internal error:
>
> ```
> "common/renaming.jazz", line 18 (2-21): internal compilation error in function mul:
>   asmgen: assemble_sopn.pseudo_op: unexpected operator mulu_32
> "common/swap.jazz", line 57 (2-21): internal compilation error in function wint:
>   asmgen: assemble_sopn.pseudo_op: unexpected operator swap
> ```

> Further tests (Stage 8) show that the 64-bit operation `hi, lo = a * 64u b` (`mulu_64`), which computes the full 128-bit product of two 64-bit numbers, fails in the same way. The x86-64 back end compiles all three operators. Because A64 provides the required instructions (`UMULH`, `UMULL`), the errors reflect a missing translation step (*lowering*) in the back end rather than an architectural limitation. They should be reported to the Jasmin developers together with the reproducers in `08-negative/`.

# 4 Stage 2: differential testing

*Purpose: detect miscompilations by comparing the results of the compiled code with the results that the language semantics prescribes.*

## 4.1 A reference interpreter for ARMv8-A

The Jasmin OCaml library contains an evaluator (module `Evaluator`) that executes a Jasmin program according to its formal semantics, but the library provides no command-line program for it, and its test driver supports only x86-64. A driver of 73 lines was therefore written. It loads a program with the *ARMv8-A* architecture definition, reads queries (a function name and argument values) from standard input, and prints the results:

```
1
2  let load_file name =
3    let open Pretyping in
4    try
5      name
6      |> tt_file Arch.arch_info Env.empty None None
7      |> fst |> Env.decls
8      |> Compile.preprocess Arch.pointer_data Arch.msf_size Arch.asmOp
9    with TyError (loc, e) ->
10     Format.eprintf "%a: %a@." Location.pp_loc loc pp_tyerror e;
11     exit 2
12
13 let value_of_arg ty s =
14   let z = Z.of_string s in
15   match ty with
16   | Prog.Bty Prog.Bool -> Values.Vbool (not (Z.equal z Z.zero))
17   | Prog.Bty (Prog.U ws) -> Values.Vword (ws, Conv.word_of_z ws z)
18   | Prog.Bty Prog.Int -> Values.Vint (Conv.cz_of_z z)
19   | _ -> failwith "unsupported argument type (only scalars)"
20
21 let string_of_value = function
22   | Values.Vbool b -> if b then "1" else "0"
23   | Values.Vword (ws, w) ->
24       "0x" ^ Z.format "%x" (Conv.z_unsigned_of_word ws w)
25   | Values.Vint z -> Z.to_string (Conv.z_of_cz z)
26   | v -> Format.asprintf "%a" Evaluator.pp_val v
27
28 let () =
29   if Array.length Sys.argv <> 2 then (
30     prerr_endline "usage: jasmin_interp FILE.jazz < queries";
31     exit 2);
32   let prog = load_file Sys.argv.(1) in
33   let funs = Hashtbl.create 17 in
34   List.iter (fun f -> Hashtbl.add funs f.Prog.f_name.fn_name f) (snd prog);
35   let uprog = Expr.to_uprog Arch.asmOp (Conv.cuprog_of_prog prog) in
36   let mem =
37     match Evaluator.initial_memory Arch.reg_size (Z.of_int 0x100000) [] with
38     | Utils0.Ok m -> m
39     | Utils0.Error _ -> failwith "cannot create initial memory"
40   in
```

```
  try
    while true do
      let line = input_line stdin in
      match String.split_on_char ' ' (String.trim line) |> List.filter (( <> ) "") with
      | [] -> ()
      | fname :: args ->
          let f = Hashtbl.find funs fname in
          let vargs = List.map2 value_of_arg f.Prog.f_tyin args in
          let out =
            match Evaluator.run (module Arch) uprog IInfo.dummy f.Prog.f_name vargs mem with
            | _, res -> String.concat " " (List.map string_of_value res)
            | exception Evaluator.Eval_error (_, err) ->
                Format.asprintf "ERROR %a" Evaluator.pp_error err
          in
          print_endline (fname ^ " " ^ out)
    done
  with End_of_file -> ()
```

Listing 1: `tools/jasmin-interp/jasmin_interp.ml` (main part)

```
$ cd tools/jasmin-interp && dune build ./jasmin_interp.exe
$ printf 'add 1 2\ndiv 100 7\ndiv 5 0\n' | ./_build/default/jasmin_interp.exe add_div.jazz
add 0xdfadcefa
div 0x2
div ERROR arithmetic error
```

Division by zero is undefined in Jasmin, and the interpreter reports it as an error. Such queries are excluded from the comparison, because the semantics prescribes no result for them (the hardware returns 0). The intrinsic `#UDIV`, which models the hardware instruction, is defined for a zero divisor, and its interpreted result agrees with the hardware.

## 4.2 The differential tester

The tester `tools/difftest.py` automates the comparison. It parses the exported functions that take scalar arguments (`reg u8..u64`) and return at most two scalar results. For these functions, it generates a C program that calls the native code; a pair of results is returned as a 16-byte structure, that is, in the registers `x0` and `x1`. The tester then generates queries, 30 % of which use edge values such as $0, 1, 2^{63}$ and $2^{64} - 1$ and the remainder uniformly distributed values, submits them to both executions, and compares the results line by line:

```
    with tempfile.TemporaryDirectory() as tmp:
        asm = os.path.join(tmp, "prog.s")
        drv = os.path.join(tmp, "driver.c")
        exe = os.path.join(tmp, "native")
        r = subprocess.run(["jasminc", "-arch", "armv8a", "-o", asm, a.jazz],
                           capture_output=True, text=True)
        if r.returncode != 0:
            sys.exit(r.stdout + r.stderr)
        open(drv, "w").write(c_driver(funcs))
        subprocess.run(["gcc", "-O1", "-o", exe, drv, asm], check=True)

        queries = gen_queries(funcs, a.n, random.Random(a.seed),
                              {k: int(v) for k, v in (b.split("=") for b in a.bits)})
        data = "\n".join(queries) + "\n"
        ref = subprocess.run([INTERP, a.jazz], input=data, capture_output=True,
                             text=True, check=True).stdout.splitlines()
        nat = subprocess.run([exe], input=data, capture_output=True,
                             text=True, check=True).stdout.splitlines()

    stats = {f: [0, 0, 0] for f in funcs}  # agree, undefined, mismatch
    shown = 0
```

```
    for q, r, n in zip(queries, ref, nat):
        f = q.split()[0]
        if " ERROR " in r + " ":
            stats[f][1] += 1
            if a.show_undef and shown < 5:
                print(f"    undefined in Jasmin: {q}  -> interpreter: {r[len(f)+1:]}, native: {n[len(f)+1:]}"
    ↪ )
                shown += 1
        elif r == n:
            stats[f][0] += 1
        else:
            stats[f][2] += 1
            print(f"    MISMATCH {q}: interpreter={r[len(f)+1:]} native={n[len(f)+1:]}")
    assert len(ref) == len(nat) == len(queries)
```

Listing 2: `tools/difftest.py`: execution of both implementations and comparison

```
$ python3 tools/difftest.py 01-upstream/armv8a/div.jazz -n 2000 --show-undef
    undefined in Jasmin: div 0x8092b4d42b28fef0 0x0  -> interpreter: ERROR arithmetic error,
    ↪ native: 0x0
  [PASS] div.jazz:div  1970 agree, 30 undefined, 0 mismatch
$ python3 tools/difftest.py 01-upstream/armv8a/loop.jazz --bits loop=12   # bound the loop
```

The option `--bits` restricts an argument to a given number of bits; it bounds, for example, the number of iterations of a loop whose trip count is an argument.

### 4.3 Instruction coverage beyond the upstream tests

The upstream tests exercise only part of the instruction set. The file `02-isa/isa.jazz` (17 functions, 264 lines) therefore covers further instructions and constructs: 128-bit carry chains, subtraction with borrow, a $64 \times 64 \to 128$ product composed of 32-bit halves, `MADD` and `MSUB` (multiply-add and multiply-subtract), constant and variable rotations (64-bit and 32-bit), masked variable shifts, logical operations with bitmask immediates, a SWAR population count (counting set bits with word-parallel arithmetic), all ten comparison conditions, truncations and sign and zero extensions, 32-bit wrap-around, loops with `UDIV` (Euclid's algorithm), nested control flow (Collatz sequence), register pressure with non-inlined calls, and stack and register arrays with dynamic indices. The subtraction relies on the carry convention of A64, in which the carry flag C after a subtraction is set when *no* borrow occurs:

```
// 128-bit subtraction. The generic borrow form (`_, x -= y - cf`) is not
// supported by this back-end, so we use the A64 carry convention
// (C = NOT borrow): a - b - borrow = a + ~b + C.
export
fn sub128(reg u64 alo ahi blo bhi) -> reg u64, reg u64 {
  reg bool n z c v;
  n, z, c, v, alo = #SUBS(alo, blo);
  bhi = !bhi;
  ahi = #ADC(ahi, bhi, c);
  return alo, ahi;
}

// Carry flag of a subtraction, as seen by Jasmin: returns 1 iff a >= b.
export
fn sub_carry(reg u64 a b) -> reg u64 {
  reg bool n z c v;
  reg u64 r one;
  n, z, c, v, a = #SUBS(a, b);
  r = 0;
  one = 1;
  r = one if c;
  return r;
```

```
23 }
```

Listing 3: `02-isa/isa.jazz`: borrow handling with the A64 carry flag

> **Finding 2: subtraction with borrow.** On ARMv8-A, the compiler rejects the generic Jasmin forms `cf, a -= c; _, b -= d - cf;` and `#sbb` with the message *"register allocation: variables {cf; _0} remain unallocated"*, and it provides no `#SBC` intrinsic. The workaround (`SUBS`; `MVN`; `ADC`) relies on the A64 convention "C = NOT borrow". Section 8 proves in EasyCrypt that the flag returned by `#SUBS` indicates $a \geq b$.

### 4.4 Independent C models

Agreement between the interpreter and the native execution cannot reveal an incorrect *algorithm*, because both execute the same source program. The program `02-isa/test_models.c` therefore compares the arithmetic functions with independent C implementations, which use the 128-bit type `unsigned __int128`, the GCC built-in function `__builtin_popcountll`, and plain C, on $10^6$ inputs; the first 169 inputs enumerate all pairs of edge values. The program `02-isa/test_ldst.c` compares the two upstream functions that access memory (`ldst`, `ldst32`) with C models. It compares the complete 32-byte buffer, so that a store to a wrong address is also detected.

| Source | Functions | Inputs each | Undefined | Mismatches |
|---|---|---|---|---|
| upstream `armv8a` (scalar) | 18 | 2 000 | 71 (division by 0) | 0 |
| `02-isa/isa.jazz` | 17 | 5 000 | 0 | 0 |
| C models (`test_models.c`) | 11 | $10^6$ | – | 0 |
| memory functions (`test_ldst.c`) | 2 | $10^6$ | – | 0 |

Table 4: Results of the differential tests. "Undefined" counts the queries excluded because the language semantics prescribes no result.

## 5 Stage 3: AAPCS64 conformance

*Purpose: verify that Jasmin functions interoperate correctly with C code, and that the erasure option removes secrets from the stack.*

A C caller relies on two rules of AAPCS64. First, a callee must preserve the registers `x19`–`x29`, the lower halves of `v8`–`v15` (`d8`–`d15`) and the stack pointer `sp` (the *callee-saved* registers): the caller may keep values in them across the call. Second, a caller may pass 32-bit arguments in 64-bit registers whose upper halves contain arbitrary values; the callee must ignore these halves. Neither rule can be tested from C, because the C compiler controls the registers. Both rules were therefore tested with a *trampoline*, a small routine written in assembly that sets up the registers, calls the Jasmin function, and inspects the registers afterwards:

```
abi_call:
        stp     x29, x30, [sp, #-176]!
        stp     x19, x20, [sp, #16]
        stp     x21, x22, [sp, #32]
        stp     x23, x24, [sp, #48]
        stp     x25, x26, [sp, #64]
        stp     x27, x28, [sp, #80]
        stp     d8,  d9,  [sp, #96]
        stp     d10, d11, [sp, #112]
        stp     d12, d13, [sp, #128]
        stp     d14, d15, [sp, #144]
        str     x3, [sp, #160]          // status pointer
        mov     x9, sp
```

```
str     x9, [sp, #168]          // SP before the call

mov     x16, x0                 // fn
mov     x0, x1
mov     x1, x2
ldr     x19, =0x1919191919191919
ldr     x20, =0x2020202020202020
```

Listing 4: `03-abi/trampoline.S`: `abi_call` (excerpt)

The routine `abi_call` loads a distinct marker value into every callee-saved register, calls the Jasmin function with `blr`, and returns a bit mask of the registers whose contents changed. The routine `abi_call_dirty32` sets the upper halves of `x0` and `x1` to `0xdeadbeef` and `0xcafebabe` before it calls a function with `u32` arguments. The routine `stack_snapshot` fills 1 KiB below `sp` with the byte `0xAA`, calls the function, and copies this area; the C test program then classifies every 64-bit word in the copy.

A function that stores secrets on the stack leaves them in memory after it returns, where later code may read them. The Jasmin compiler can erase the stack frame before returning (*stack zeroization*), in a loop or with unrolled stores. The program `03-abi/stackzero.jazz` distributes a secret key over a 256-byte stack buffer, in three variants: without zeroization, with `#[stackzero=loop]`, and with `#[stackzero=unrolled]`.

| Check | Functions | Result |
|---|---|---|
| x19–x29, d8–d15 and sp preserved | 8 (including 26 live values and calls) | 0 violations in 160 000 calls |
| undefined upper halves of 32-bit arguments | 6 | 0 differences in 1 200 000 calls |

| Stack after return | Secret words | Return address | Zero words | Other |
|---|---|---|---|---|
| without zeroization | 31 | 1 | 1 | 0 |
| `stackzero=loop` | **0** | 1 | 32 | 0 |
| `stackzero=unrolled` | **0** | 1 | 32 | 0 |

Table 5: Results of the ABI and stack-zeroization tests.

The single remaining word is the link register (the return address), which the function saves with `str x30, [sp, #-16]!`; it is a public code address outside the erased frame. Without zeroization, 31 of the 32 secret-derived words remain readable after the call; the 32nd word equals $k \oplus k = 0$.

> **Note.** The compiler rejects `#[stackzero=loopSCT]`, the variant that is hardened against speculative execution, on ARMv8-A: *"Strategy "loop with SCT" is not supported in ARMv8-A"*.

# 6 Stage 4: cryptographic kernels

*Purpose: verify that the back end compiles complete cryptographic algorithms correctly, by comparison with standardized test vectors and with an established library.*

ChaCha20 is a stream cipher, and Poly1305 is a message authentication code; RFC 8439 standardizes both, and they are widely used together (for example in TLS). Both were implemented in Jasmin for this stage.

## 6.1 ChaCha20

The ChaCha20 block function transforms a state of 16 words of 32 bits in 20 rounds. The Jasmin implementation holds the state in 32-bit (W) registers. The 32-bit rotations compile to the instruction `ror`, and the pointer arguments are stored on the stack during the 20 rounds:

```
// ChaCha20 block function (RFC 8439, section 2.3) for ARMv8-A.
//   out[0..64) = ChaCha20_block(key[0..32), counter, nonce[0..12))
// The 16-word state lives in 32-bit (W) registers; the pointers are kept on
// the stack during the 20 rounds to leave room for the state.

inline
fn qr(reg u32[16] x, inline int a b c d) -> reg u32[16] {
  x[a] += x[b]; x[d] ^= x[a]; x[d] = x[d] <<r 16;
  x[c] += x[d]; x[b] ^= x[c]; x[b] = x[b] <<r 12;
  x[a] += x[b]; x[d] ^= x[a]; x[d] = x[d] <<r 8;
  x[c] += x[d]; x[b] ^= x[c]; x[b] = x[b] <<r 7;
  return x;
}

inline
fn init(reg u64 key nonce, reg u32 counter) -> reg u32[16] {
  reg u32[16] x;
  inline int i;
  x[0] = 0x61707865; x[1] = 0x3320646e; x[2] = 0x79622d32; x[3] = 0x6b206574;
  for i = 0 to 8 { x[4 + i] = [:u32 key + 4 * i]; }
  x[12] = counter;
  for i = 0 to 3 { x[13 + i] = [:u32 nonce + 4 * i]; }
  return x;
}

#[ct = "public * public * public * public -> ()"]  // pointers are public; memory is secret
export
fn chacha20_block(reg u64 out key nonce, reg u32 counter) {
  stack u64 s_out s_key s_nonce;
  stack u32 s_counter;
  reg u32[16] x y;
  reg u32 v;
  reg u64 round;
  inline int i;

  s_out = out; s_key = key; s_nonce = nonce; s_counter = counter;
  x = init(key, nonce, counter);

  round = 10;
  while (round != 0) {
    x = qr(x, 0, 4,  8, 12);   // column rounds
    x = qr(x, 1, 5,  9, 13);
    x = qr(x, 2, 6, 10, 14);
    x = qr(x, 3, 7, 11, 15);
    x = qr(x, 0, 5, 10, 15);   // diagonal rounds
    x = qr(x, 1, 6, 11, 12);
    x = qr(x, 2, 7,  8, 13);
    x = qr(x, 3, 4,  9, 14);
    round -= 1;
  }
```

Listing 5: `04-crypto/chacha20.jazz` (rounds; the feed-forward follows)

## 6.2 Poly1305 without a wide multiplier

Poly1305 evaluates a polynomial modulo the prime $2^{130} - 5$. The usual 64-bit implementation represents 130-bit numbers in three 64-bit limbs and requires $64 \times 64 \to 128$ products. Because `mulu` causes an internal compiler error (Finding 1), the implementation instead represents the accumulator in radix $2^{26}$, that is, in five limbs of 26 bits. Every partial product then fits into a 64-bit register, and `MADD` (multiply-add) accumulates it:

```
inline
```

```
fn block(reg u64[5] h r, reg u64[4] s, reg u64 lo hi hibit) -> reg u64[5] {
  reg u64[5] b d;
  reg u64 c;
  inline int i;

  b = split(lo, hi);
  b[4] |= hibit;
  for i = 0 to 5 { h[i] += b[i]; }

  // s[i] = 5 * r[i + 1]: terms that wrap around 2^130 are multiplied by 5.
  d[0] = h[0] * r[0]; d[0] += h[1] * s[3]; d[0] += h[2] * s[2]; d[0] += h[3] * s[1]; d[0] += h[4] * s[0];
  d[1] = h[0] * r[1]; d[1] += h[1] * r[0]; d[1] += h[2] * s[3]; d[1] += h[3] * s[2]; d[1] += h[4] * s[1];
  d[2] = h[0] * r[2]; d[2] += h[1] * r[1]; d[2] += h[2] * r[0]; d[2] += h[3] * s[3]; d[2] += h[4] * s[2];
  d[3] = h[0] * r[3]; d[3] += h[1] * r[2]; d[3] += h[2] * r[1]; d[3] += h[3] * r[0]; d[3] += h[4] * s[3];
  d[4] = h[0] * r[4]; d[4] += h[1] * r[3]; d[4] += h[2] * r[2]; d[4] += h[3] * r[1]; d[4] += h[4] * r[0];

  // carry propagation
  c = d[0] >> 26; h[0] = d[0]; h[0] &= MASK26; d[1] += c;
  c = d[1] >> 26; h[1] = d[1]; h[1] &= MASK26; d[2] += c;
  c = d[2] >> 26; h[2] = d[2]; h[2] &= MASK26; d[3] += c;
  c = d[3] >> 26; h[3] = d[3]; h[3] &= MASK26; d[4] += c;
  c = d[4] >> 26; h[4] = d[4]; h[4] &= MASK26;
  h[0] += c; c = c << 2; h[0] += c;          // h0 += 5 * c (MUL has no immediate)
  c = h[0] >> 26; h[0] &= MASK26; h[1] += c;
  return h;
}
```

Listing 6: `04-crypto/poly1305.jazz`: processing of one block

The final reduction (subtraction of $2^{130} - 5$ if $h \geq 2^{130} - 5$) is a branch-free selection with a mask. Loops with a public length assemble the last partial block of the message byte by byte.

### 6.3 Results

| Kernel | Oracle | Result |
|---|---|---|
| ChaCha20 block | RFC 8439 §2.3.2 test vector | ✓ |
| ChaCha20 block | OpenSSL `EVP_chacha20`: random key and nonce; counters 0, $2^{32} - 1$ and random | 200 000 / 200 000 |
| Poly1305 | RFC 8439 §2.5.2 test vector | ✓ |
| Poly1305 | OpenSSL `EVP_MAC("POLY1305")`: lengths 0..300; random, 0x00 and 0xff data | 62 608 / 62 608 |

Table 6: Cryptographic kernels compared with RFC 8439 and OpenSSL 3.0.20.

Messages and keys that consist of `0xff` bytes bring the accumulator close to the modulus and thereby exercise the final conditional subtraction, which random data rarely reach.

## 7 Stage 5: constant time

*Purpose: verify that the constant-time checker classifies constant-time and leaky code correctly on ARMv8-A, and that the processor behaves accordingly.*

### 7.1 Static verification with `jasmin-ct`

`jasmin-ct` offers three modes. The default mode (CT) checks that secrets influence neither branches nor memory addresses. The mode `--doit` additionally requires that secrets be processed only by instructions whose timing the architecture guarantees to be independent of their operands

(data-operand-independent timing, DOIT). The mode `--sct` checks speculative constant time; programs achieve this property with speculative load hardening.

The file `05-ct/ct.jazz` pairs three constant-time kernels with leaky counterparts: buffer comparison (without and with early exit), conditional swap (masking and branching), and table lookup (scan of the whole table with `CSEL` and direct indexing with the secret index). A function that divides by a secret completes the set. Annotations declare the security types. Pointers are public, and the memory to which they point is secret by default:

```
// ---------------------------------------------------- table look-up
// Returns table[idx] (16 entries) by scanning the whole table.
#[ct = "public * secret -> secret"]
export
fn ct_lookup16(reg u64 table idx) -> reg u64 {
  reg u64 r v i;
  reg bool eq;
  r = 0;
  i = 0;
  while (i < 16) {
    v = [table + 8 * i];
    eq = i == idx;
    r = v if eq;               // CSEL: data-independent timing
    i += 1;
  }
  return r;
}

// Direct indexing with a secret index: the address depends on the secret.
#[ct = "public * secret -> secret"]
export
fn leaky_lookup16(reg u64 table idx) -> reg u64 {
  reg u64 r;
  idx &= 15;
  r = [table + 8 * idx];
  return r;
}

// Division by a secret: UDIV timing may depend on its operands.
```

Listing 7: `05-ct/ct.jazz`: table lookup by scan and by direct indexing

| **Function** | `jasmin-ct` | `--doit` | `--sct` |
|---|---|---|---|
| `ct_memeq`, `ct_cswap4`, `ct_lookup16` | ✓ | ✓ | × (no SLH) |
| `chacha20_block`, `poly1305` | ✓ | ✓ | × (no SLH) |
| `leaky_memeq` (secret branch) | × | | |
| `leaky_cswap4` (secret branch) | × | | |
| `leaky_lookup16` (secret address) | × | | |
| `leaky_div` (secret divisor) | × | | |
| `sct_memeq`, `sct_lookup16` (`sct.jazz`, SLH) | | | ✓ |

Table 7: Verdicts of `jasmin-ct --arch=armv8a` (✓ accepted, × rejected); all verdicts are as expected.

**Finding 3: no speculative load hardening on ARMv8-A.** The file `05-ct/sct.jazz` hardens `memeq` and `lookup16` with SLH: `#init_msf` and `#update_msf` maintain a *misspeculation flag*, which records whether the processor is executing a mispredicted branch, and `#protect` masks values with this flag so that misspeculated loads cannot leak secrets. `jasmin-ct --sct --arch=armv8a` accepts the file, and `jasminc -arch x86-64` compiles it, but the ARMv8-A back end rejects it:

```
$ jasminc -arch armv8a 05-ct/sct.jazz
```

```
speculative execution lowering: The architecture does not provides protection
for selective speculative load hardening
```

All 24 upstream SLH tests (`tests/success/slh`) fail in the same way on ARMv8-A. Protection against Spectre v1 is therefore not yet available for this target, although AArch64 provides the necessary means (`CSDB` and conditional selection). In addition, the SLH lowering recognizes only loop conditions that are bound to a Boolean variable, as in `while {c = i < n;} (c)`.

### 7.2 Timing measurements

The static checks concern the program; the actual timing behavior depends on the processor. The timing of the hardware was therefore measured with the method of the tool *dudect*: two classes of inputs (for example, two equal buffers and two buffers that differ in the first byte) are interleaved at random, each execution is timed, and the two distributions of execution times are compared with Welch's $t$-test. The timer is the ARMv8 virtual counter `CNTVCT_EL0` (54 MHz), read after the barrier instruction `isb`. Following the threshold customary in leakage assessment (TVLA), a statistic $|t| > 4.5$ is taken as evidence of a timing difference, that is, of a leak.

```c
static inline uint64_t ticks(void) {
  uint64_t t;
  __asm__ volatile("isb\n\tmrs %0, cntvct_el0" : "=r"(t) :: "memory");
  return t;
}
static uint64_t freq(void) {
  uint64_t f;
  __asm__ volatile("mrs %0, cntfrq_el0" : "=r"(f));
  return f;
}

/* Welch's t statistic, computed online. */
typedef struct { double n[2], mean[2], m2[2]; } welch_t;
static void welch_add(welch_t *w, int cls, double x) {
  w->n[cls]++;
  double d = x - w->mean[cls];
  w->mean[cls] += d / w->n[cls];
  w->m2[cls] += d * (x - w->mean[cls]);
}
static double welch_t_value(const welch_t *w) {
  double v0 = w->m2[0] / (w->n[0] - 1), v1 = w->m2[1] / (w->n[1] - 1);
  return (w->mean[0] - w->mean[1]) / sqrt(v0 / w->n[0] + v1 / w->n[1]);
}
```

Listing 8: `05-ct/test_ct.c`: timer and Welch statistic

| Function | Class 0 | Class 1 | Ticks (class 0) | Ticks (class 1) | $\lvert t\rvert$ |
|---|---|---|---|---|---|
| `ct_memeq` (4 KiB) | equal | differ at byte 0 | 919.3 | 907.2 | 0.65 |
| `leaky_memeq` | equal | differ at byte 0 | 1504.8 | 2.3 | **408.77** |
| `ct_cswap4` ($\times 256$) | $c = 0$ | $c = 1$ | 72.2 | 72.1 | 0.45 |
| `leaky_cswap4` ($\times 256$) | $c = 0$ | $c = 1$ | 24.9 | 56.2 | **65.25** |

Table 8: Timing tests (20 000 samples per class; ticks of the 54 MHz counter). The verdicts agree with those of `jasmin-ct`.

## 8 Stage 6: EasyCrypt

*Purpose: verify that the translation of ARMv8-A programs to EasyCrypt works, and prove constant time and functional correctness formally.*

## 8.1 Extraction

Each of the five test programs is translated twice for ARMv8-A: once as a plain EasyCrypt module, and once with the option `-m CT`, which produces a *leakage model*: every procedure additionally returns its leakage (branch conditions, memory addresses, loop bounds) as part of its result. All ten files type-check against the eclib model `JModel_armv8a.ec`:

```
$ jasmin2ec --arch=armv8a       -o 06-easycrypt/extracted/Ct.ec    05-ct/ct.jazz
$ jasmin2ec --arch=armv8a -m CT -o 06-easycrypt/extracted/Ct_CT.ec 05-ct/ct.jazz
$ cd 06-easycrypt/extracted && easycrypt compile -no-eco Ct_CT.ec
```

| Program | Lines (plain model) | Lines (leakage model) |
|---|---|---|
| isa.jazz | 390 | 494 |
| stackzero.jazz | 48 | 75 |
| chacha20.jazz | 134 | 272 |
| poly1305.jazz | 293 | 583 |
| ct.jazz | 115 | 221 |

Table 9: Extracted EasyCrypt models (each model type-checks in about 10 s).

## 8.2 Constant-time proofs

Constant time is a *2-safety* property: it relates two executions rather than one. Two executions that agree on the public inputs must produce the same leakage, *regardless of the secrets*. In pRHL, which relates two programs, this property is an `equiv` judgment whose precondition equates only the public arguments and whose postcondition equates the leakage. The precondition does not equate the memory (`Glob.mem`), so keys and messages may differ between the two executions:

```
(* Constant-time security of the Jasmin kernels, proved on the leakage
   model produced by `jasmin2ec -m CT` (each procedure returns, as first
   component, the trace of branch conditions and memory addresses).

   Non-interference statement: two runs that agree on the public inputs
   produce the same leakage, whatever the secrets -- the memory contents
   (Glob.mem) and the secret arguments are NOT assumed equal. *)
require import AllCore.
from Jasmin require import JModel_armv8a.
require Ct_CT Chacha20_CT Poly1305_CT.

lemma ct_memeq_ct :
  equiv [ Ct_CT.M.ct_memeq ~ Ct_CT.M.ct_memeq :
          ={a, b, len} ==> res{1}.`1 = res{2}.`1 ].
proof.
(* the loaded bytes differ, but the trace only depends on i, len, a, b *)
proc; wp; while (={i, len, a, b, leak_cond, _b}); by auto.
qed.

(* idx is secret *)
lemma ct_lookup16_ct :
  equiv [ Ct_CT.M.ct_lookup16 ~ Ct_CT.M.ct_lookup16 :
          ={table} ==> res{1}.`1 = res{2}.`1 ].
proof.
(* the comparison i = idx feeds a CSEL, not a branch: it is not leaked *)
proc; wp; while (={i, table, leak_cond, _b}); by auto.
qed.

(* the swap bit c is secret *)
lemma ct_cswap4_ct :
  equiv [ Ct_CT.M.ct_cswap4 ~ Ct_CT.M.ct_cswap4 :
```

```
          ={p, q} ==> ={res} ].
proof. by proc; sim. qed.

(* the key (in memory) is secret; pointers and counter are public *)
lemma chacha20_ct :
  equiv [ Chacha20_CT.M.chacha20_block ~ Chacha20_CT.M.chacha20_block :
          ={out, key, nonce, counter} ==> ={res} ].
proof. by proc; inline *; sim. qed.

(* the key and the message (in memory) are secret; len is public *)
lemma poly1305_ct :
  equiv [ Poly1305_CT.M.poly1305 ~ Poly1305_CT.M.poly1305 :
          ={out, m, len, key} ==> ={res} ].
proof. by proc; inline *; sim. qed.
```

Listing 9: `06-easycrypt/proofs/ct_proofs.ec`

All five proofs are checked in 11 s. For `ct_lookup16`, the proof confirms that the comparison `i = idx` is an operand of a `CSEL`, not a branch condition, and therefore does not appear in the leakage. For the three leaky functions, the same proof scripts must fail, and they do (`proofs/fail_leaky_*.ec`):

```
$ easycrypt compile -no-eco -I extracted proofs/fail_leaky_memeq.ec
[critical] [proofs/fail_leaky_memeq.ec: line 11 (0-58)] [by]: cannot close goals
```

## 8.3 Functional correctness of the carry handling

The ARMv8-A model defines `SUBS` (subtraction that sets the flags) as `ADCS x (invw y) true`, that is, as an addition of the bitwise complement with an incoming carry, in accordance with the A64 carry convention. The proofs establish that the 128-bit addition of `isa.jazz` is exact modulo $2^{128}$ and that the flag returned by `#SUBS` is set exactly when no borrow occurs. These results justify the workaround of Finding 2:

```
(* ------------------------------------------------------------------ *)
(* 128-bit addition with ADDS / ADCS                                  *)

op val128 (lo hi : W64.t) = to_uint lo + W64.modulus * to_uint hi.

lemma add128_correct a0 a1 b0 b1 :
  hoare [ Isa.M.add128 : alo = a0 /\ ahi = a1 /\ blo = b0 /\ bhi = b1
          ==> val128 res.`1 res.`2 = (val128 a0 a1 + val128 b0 b1) %% W64.modulus ^ 2 ].
proof.
proc; wp; skip => />.
have hc : b2i (carry_add a0 b0 false) = (to_uint a0 + to_uint b0) %/ W64.modulus.
  rewrite /carry_add /b2i /=; apply carry_div => //.
  by move: (W64.to_uint_cmp a0) (W64.to_uint_cmp b0) => /= ? ?; smt().
rewrite /val128 !W64.addcE /= hc /b2i /= !W64.to_uintD /=.
have -> : to_uint a0 + 18446744073709551616 * to_uint a1 + (to_uint b0 + 18446744073709551616 * to_uint b1)
        = (to_uint a0 + to_uint b0) + 18446744073709551616 * (to_uint a1 + to_uint b1) by ring.
have -> : 340282366920938463463374607431768211456 = 18446744073709551616 * 18446744073709551616 by done.
by rewrite mod_split // W64.of_uintK /= modzDml modzDmr.
qed.

(* ------------------------------------------------------------------ *)
(* The A64 carry flag of a subtraction means "no borrow"              *)

lemma sub_carry_correct a0 b0 :
  hoare [ Isa.M.sub_carry : a = a0 /\ b = b0
          ==> res = if to_uint b0 <= to_uint a0 then W64.one else W64.zero ].
proof.
proc; wp; skip => />.
rewrite /SUBS_64 /ADCS_64 /ADC_64 /nzcv_64 /b2i /=.
rewrite !W64.to_uintD W64.to_uint_invw /= modzDml.
```

```
move: (W64.to_uint_cmp a0) (W64.to_uint_cmp b0) => /= ha hb.
have -> : to_uint a0 + (18446744073709551615 - to_uint b0) + 1
        = (to_uint a0 - to_uint b0) + 18446744073709551616 by ring.
case: (to_uint b0 <= to_uint a0) => h.
+ by rewrite modzDr modz_small; smt().
+ by rewrite modz_small; smt().
qed.
```

Listing 10: `06-easycrypt/proofs/isa_correct.ec` (main lemmas)

The two auxiliary integer lemmas (`mod_split`, `carry_div`) are proved in the same file, which is checked in 14 s.

> **Note.** When a tactic fails, `easycrypt compile` prints only the error. To inspect proof goals without an editor, pass a prefix of the file to `easycrypt cli -emacs -I extracted`. The proofs of this stage were developed in this way on the Raspberry Pi.

## 9 Stage 7: performance

*Purpose: compare the speed of the code generated by the ARMv8-A back end with that of C and of optimized libraries.*

The program `07-bench/bench.c` measures throughput as the median of 11 runs and reports it in nanoseconds per byte and in processor cycles per byte at 2.4 GHz (lower values are better). For comparison, it measures portable C implementations of the *same algorithms* (scalar ChaCha20, Poly1305 in radix $2^{26}$) and OpenSSL, which uses hand-written NEON (vector) and scalar assembly:

| Kernel | Implementation | ns/byte | Cycles/byte |
|---|---|---|---|
| ChaCha20, 1 block | Jasmin (scalar) | 2.86 | 6.86 |
| | C, `gcc -O2` (scalar) | 2.50 | 6.01 |
| | OpenSSL, 64-byte calls | 2.64 | 6.33 |
| ChaCha20, 4 KiB | OpenSSL (NEON, several blocks) | 1.00 | 2.40 |
| Poly1305, 4 KiB | Jasmin (radix $2^{26}$, 64-bit `MADD`) | 2.27 | 5.45 |
| | C, `gcc -O2` (radix $2^{26}$, `UMADDL`) | 0.91 | 2.17 |
| | OpenSSL | 0.50 | 1.21 |

Table 10: Throughput on the Cortex-A76. The Jasmin results do not depend on the C compiler flags; with `-O3`, the C results differ by less than 3 %.

The scalar ChaCha20 implementation in Jasmin is within 15 % of the code generated by GCC. A faster implementation would require NEON, which this release of the back end does not support (it has no vector registers).

> **Finding 4: UMULL and UMADDL modeled but not available.** The inner loop of Poly1305 comprises about 90 instructions per 16 bytes and causes no stack traffic. The difference from C originates in the multiplications: GCC emits 25 `umaddl` and `umull` instructions, which multiply two 32-bit values into a 64-bit result, whereas Jasmin can emit only the 64-bit `madd`, which has a longer latency on the Cortex-A76. The eclib model already defines these instructions (`op UMULL` and `op UMADDL` in `JModel_armv8a.ec`), but the compiler rejects them with the message *"unknown primitive: "UMULL""*. Making them available, together with `UMULH` for `mulu`, would remove most of the difference.

## 10 Stage 8: limitations of the back end

*Purpose: record every limitation found during the campaign in a form that detects its removal in a future release.*

Each limitation has a minimal reproducer in `08-negative/`: a small program that the back end must reject. Each reproducer states the expected error message in a comment line that begins with `// EXPECT:`, and `tools/negtest.sh` verifies that `jasminc -arch armv8a` rejects the program with a matching message. If a future release removes a limitation, the corresponding program compiles, and the test fails; the change is therefore noticed. For comparison, the last column of the table indicates whether the x86-64 back end accepts the same program.

| Test | Limitation (ARMv8-A) | Workaround | x86 |
|---|---|---|---|
| `mulu64` | ICE: $64 \times 64 \to 128$ `hi, lo = a * 64u b` | radix $2^{26}$ or 32-bit halves | ✓ |
| `mulu32` | ICE: $32 \times 32 \to 64$ into two registers | `(64u)` casts | ✓ |
| `swap32` | ICE: `#swap` on `u32` (the 64-bit form works) | 64-bit swap | ✓ |
| `sbb` | generic borrow: the flag cannot be allocated | `SUBS`+`MVN`+`ADC` | ✓ |
| `slh` | no SLH (`init_msf`, `protect`) | none | ✓ |
| `stackzero_-loopsct` | `stackzero=loopSCT` not supported | `loop`, `unrolled` | ✓ |
| `umull` | `UMULL` and `UMADDL` are not intrinsics (but are defined in eclib) | 64-bit `MUL` | – |
| `vector` | no vector (NEON) registers | scalar code | ✓ |
| `u8_arith` | no 8-bit or 16-bit registers | 32-bit or 64-bit operations | ✓ |
| `args9` | at most 8 arguments (no arguments on the stack) | pass a pointer | × |
| `adc_imm` | `ADC` has no immediate form | register | ✓ |
| `mul_imm` | `MUL` has no immediate form | register | ✓ |
| `add32_imm` | 32-bit `ADD`: only 12-bit immediates (handled automatically for 64-bit) | register | × |
| `logic_imm` | logical operations: only bitmask immediates | register | ✓ |
| `csel_imm` | conditional move of an immediate | register | × |
| `shift_unmasked` | a variable shift must have the form `x « (n & 63)` | mask | × |
| `rotl_var` | no rotation to the left by a register | `»r` by $-n$ | × |

Table 11: Expected compilation failures (all 17 programs are rejected with the expected message). ICE denotes an internal compiler error. In the x86 column, ✓ means that the x86-64 back end accepts the program and × that it rejects it as well.

The upper part of the table lists features that the back end lacks. The lower part follows from the A64 instruction set or from Jasmin's deliberate rule that each source instruction corresponds to exactly one machine instruction. These restrictions are intended, but users familiar with x86-64 may not expect them: on x86-64, for example, `a * 3` and `_, c += 0 + cf` are valid.

## 11 Conclusion and recommendations

On the Raspberry Pi 5, the ARMv8-A back end of Jasmin 2026.09.0 generated correct code for every test program that it accepted. This result covers the 72 upstream programs, the 17 instruction-coverage functions, the ABI tests and two complete RFC 8439 primitives. In more than $10^7$ executions, no result disagreed with the language semantics or with OpenSSL. The generated code conforms to the calling convention, stack zeroization is effective, and the constant-time guarantees hold statically, empirically and in EasyCrypt proofs on the extracted leakage model.

The back end is nevertheless at an early stage of development. The following actions are recommended:

1. **Report** the three internal compiler errors (`08-negative/mulu64.jazz`, `mulu32.jazz`, `swap32.jazz`) and the register-allocation failure for borrows (`sbb.jazz`) to the Jasmin developers.
2. Do **not** yet rely on protection against Spectre v1 on ARMv8-A: the checker accepts SLH programs that the compiler cannot compile.
3. Use the workarounds of Stage 8 for wide multiplications and borrows. This test suite covers their correctness, and an EasyCrypt proof additionally covers the borrow.
4. Rerun `run-all.sh` after each upgrade of Jasmin. The expected-failure tests detect every limitation that a new release removes.

The third document of this repository (`../NTRU+`) uses a development version of the back end with NEON support, which removes the limitation `vector` of Stage 8, to implement the vector kernels of a post-quantum key encapsulation mechanism.

# A Glossary

**A64** The instruction set of AArch64.

**Borrow** The bit that a subtraction takes from the next higher limb when its result would be negative.

**Callee-saved register** A register whose value a called function must restore before it returns.

**dudect** A method and tool for detecting timing leaks by statistical comparison of the execution times of two input classes.

**eclib** The EasyCrypt library of Jasmin, which models machine words and instructions.

**Extraction** The translation of a Jasmin program into an EasyCrypt module by `jasmin2ec`.

**Immediate** A constant operand encoded directly in an instruction.

**Intrinsic** A Jasmin operator that corresponds to one specific machine instruction, written with the prefix `#`.

**Lowering** A compiler step that replaces a high-level operation by machine instructions.

**Radix $2^{26}$** A representation of a large integer by limbs of 26 bits each.

**Reproducer** A minimal program that triggers a specific behavior of the compiler.

**Stack zeroization** The erasure of a function's stack frame before the function returns.

**Trampoline** A small assembly routine that sets up the processor state, calls another function, and inspects the state afterwards.

**2-safety** A property that relates two executions of the same program, such as constant time.

# B Complete output of `run-all.sh`

```
== Environment
   Raspberry Pi 5 Model B Rev 1.0, Linux 6.12.93+rpt-rpi-2712 aarch64
   Jasmin Compiler 2026.09.0, EasyCrypt r2026.07
   gcc (Debian 12.2.0-14+deb12u1) 12.2.0, OpenSSL 3.0.20 7 Apr 2026 (Library: OpenSSL 3.0.20 7 Apr 2026)

== Stage 1: upstream Jasmin tests on ARMv8-A (compile, assemble, CT, EasyCrypt)
   started 23:35:56
  [PASS] upstream armv8a: 16/16 compiled with -arch armv8a and assembled by GNU as
  [PASS] upstream stack_zeroization: 1/1 compiled with -arch armv8a and assembled by GNU as
  [PASS] upstream common: 51/55 compiled with -arch armv8a and assembled by GNU as, 4 known internal compiler errors (
     ↪ mulu_32, swap_32)
  [PASS] jasmin-ct accepts add_poly.jazz
  [PASS] jasmin-ct accepts bug_746.jazz
  [PASS] jasmin-ct accepts declassify_load.jazz
  [PASS] jasmin-ct accepts div.jazz
  [PASS] jasmin-ct accepts doc_add2.jazz
```

```
  [PASS] jasmin-ct accepts doc_add_SP.jazz
  [PASS] jasmin-ct accepts doc_add_restr.jazz
  [PASS] jasmin-ct accepts doc_load_mem.jazz
  [PASS] jasmin-ct accepts doc_set0_incr.jazz
  [PASS] jasmin-ct accepts doc_store.jazz
  [PASS] jasmin-ct accepts doc_zero.jazz
  [PASS] jasmin-ct accepts otp_declassify_if.jazz
  [PASS] jasmin-ct accepts randombytes.jazz
  [PASS] jasmin-ct accepts sct.jazz
  [PASS] jasmin-ct accepts unused_poly.jazz
  [PASS] jasmin-ct rejects H_L_L.jazz
  [PASS] jasmin-ct rejects div_secret.jazz
  [PASS] jasmin-ct rejects doc_bad_call.jazz
  [PASS] jasmin-ct rejects doc_leak_arr_load.jazz
  [PASS] jasmin-ct rejects doc_leak_cond.jazz
  [PASS] jasmin-ct rejects doc_leak_store.jazz
  [PASS] jasmin-ct rejects load_on_poly.jazz
  [PASS] jasmin-ct rejects load_on_secret.jazz
  [PASS] jasmin-ct rejects mod_secret.jazz
  [PASS] jasmin-ct rejects otp_if.jazz
  [PASS] jasmin-ct rejects randombytes.jazz
  [PASS] jasmin-ct rejects secret_ptr.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of add.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of addcarry.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of and.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of call.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of csel.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of div.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of intrinsics.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of ldst.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of loop.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of mul.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of or.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of shift.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of sub.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of swap.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of unaligned_stackalign.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of word32.jazz
  [PASS] jasmin2ec + EasyCrypt type-check of stack-zeroization.jazz

== Stage 2: differential testing: reference interpreter vs native execution
   started 23:38:52
  [PASS] add.jazz:add  2000 agree, 0 mismatch
  [PASS] addcarry.jazz:addcarry  2000 agree, 0 mismatch
  [PASS] and.jazz:and  2000 agree, 0 mismatch
  [PASS] call.jazz:call  2000 agree, 0 mismatch
  [PASS] csel.jazz:csel  2000 agree, 0 mismatch
  [PASS] div.jazz:div  1970 agree, 30 undefined, 0 mismatch
  [PASS] intrinsics.jazz:intrinsics  2000 agree, 0 mismatch
  [PASS] loop.jazz:loop  2000 agree, 0 mismatch
  [PASS] mul.jazz:mul  2000 agree, 0 mismatch
  [PASS] or.jazz:or  2000 agree, 0 mismatch
  [PASS] shift.jazz:shift  2000 agree, 0 mismatch
  [PASS] sub.jazz:sub  2000 agree, 0 mismatch
  [PASS] swap.jazz:swap  2000 agree, 0 mismatch
  [PASS] unaligned_stackalign.jazz:instack  2000 agree, 0 mismatch
  [PASS] word32.jazz:arith32  1959 agree, 41 undefined, 0 mismatch
  [PASS] word32.jazz:logic32  2000 agree, 0 mismatch
  [PASS] word32.jazz:shift32  2000 agree, 0 mismatch
  [PASS] word32.jazz:csel32  2000 agree, 0 mismatch
  [PASS] isa.jazz:add128  5000 agree, 0 mismatch
  [PASS] isa.jazz:sub128  5000 agree, 0 mismatch
  [PASS] isa.jazz:sub_carry  5000 agree, 0 mismatch
  [PASS] isa.jazz:mul64x64  5000 agree, 0 mismatch
  [PASS] isa.jazz:madd_msub  5000 agree, 0 mismatch
  [PASS] isa.jazz:rot64  5000 agree, 0 mismatch
  [PASS] isa.jazz:rot32  5000 agree, 0 mismatch
  [PASS] isa.jazz:shifts  5000 agree, 0 mismatch
  [PASS] isa.jazz:logic  5000 agree, 0 mismatch
  [PASS] isa.jazz:popcount  5000 agree, 0 mismatch
  [PASS] isa.jazz:compare  5000 agree, 0 mismatch
  [PASS] isa.jazz:casts  5000 agree, 0 mismatch
  [PASS] isa.jazz:word32_wrap  5000 agree, 0 mismatch
  [PASS] isa.jazz:gcd  5000 agree, 0 mismatch
  [PASS] isa.jazz:collatz  5000 agree, 0 mismatch
```

```
  [PASS] isa.jazz:pressure  5000 agree, 0 mismatch
  [PASS] isa.jazz:arrays  5000 agree, 0 mismatch
  [PASS] isa models: 1000000 tests
  [PASS] upstream ldst/ldst32 vs C model: 1000000 tests

== Stage 3: AAPCS64 conformance: callee-saved registers, 32-bit arguments, stack zeroization
   started 23:40:02
  [PASS] callee-saved pressure     x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved gcd          x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved arrays       x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved collatz      x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved compare      x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved sz_none      x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved sz_loop      x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] callee-saved sz_unrolled  x19-x29, d8-d15, sp preserved (20000 calls)
  [PASS] dirty upper halves  arith32      0/200000 results differ
  [PASS] dirty upper halves  logic32      0/200000 results differ
  [PASS] dirty upper halves  shift32      0/200000 results differ
  [PASS] dirty upper halves  csel32       0/200000 results differ
  [PASS] dirty upper halves  rot32        0/200000 results differ
  [PASS] dirty upper halves  word32_wrap  0/200000 results differ
  [PASS] stack after sz_none      secret words: 31, return addresses: 1, zeroed words:  1, other: 0
  [PASS] stack after sz_loop      secret words:  0, return addresses: 1, zeroed words: 32, other: 0
  [PASS] stack after sz_unrolled  secret words:  0, return addresses: 1, zeroed words: 32, other: 0

== Stage 4: cryptographic kernels: ChaCha20 and Poly1305 vs RFC 8439 and OpenSSL
   started 23:40:03
  [PASS] chacha20_block: RFC 8439 2.3.2 test vector
  [PASS] chacha20_block vs OpenSSL EVP_chacha20: 200000 tests
  [PASS] poly1305: RFC 8439 2.5.2 test vector
  [PASS] poly1305 vs OpenSSL EVP_MAC (lengths 0..300): 62608 tests

== Stage 5: constant time: jasmin-ct, speculative CT, and timing measurements
   started 23:40:05
  [PASS] jasmin-ct accepts ct_memeq
  [PASS] jasmin-ct --doit accepts ct_memeq
  [PASS] jasmin-ct accepts ct_cswap4
  [PASS] jasmin-ct --doit accepts ct_cswap4
  [PASS] jasmin-ct accepts ct_lookup16
  [PASS] jasmin-ct --doit accepts ct_lookup16
  [PASS] jasmin-ct rejects leaky_memeq
  [PASS] jasmin-ct rejects leaky_cswap4
  [PASS] jasmin-ct rejects leaky_lookup16
  [PASS] jasmin-ct rejects leaky_div
  [PASS] jasmin-ct accepts chacha20
  [PASS] jasmin-ct --doit accepts chacha20
  [PASS] jasmin-ct --sct rejects chacha20 (no SLH protections)
  [PASS] jasmin-ct accepts poly1305
  [PASS] jasmin-ct --doit accepts poly1305
  [PASS] jasmin-ct --sct rejects poly1305 (no SLH protections)
  [PASS] jasmin-ct --sct accepts the SLH-hardened sct.jazz
  [PASS] jasminc -arch armv8a rejects sct.jazz (SLH not implemented)
  [PASS] jasminc -arch x86-64 compiles sct.jazz
  [PASS] ct_memeq, ct_cswap4, ct_lookup16 (+ leaky versions) vs C: 200000 tests
  timing: CNTVCT_EL0 at 54.0 MHz, 20000 samples per class
  [PASS] timing ct_memeq      mean ticks    919.3 vs    907.2  |t| =     0.65  -> no leak detected (expected no leak)
  [PASS] timing leaky_memeq   mean ticks   1504.8 vs      2.3  |t| =   408.77  -> leak (expected leak)
  [PASS] timing ct_cswap4     mean ticks     72.2 vs     72.1  |t| =     0.45  -> no leak detected (expected no leak)
  [PASS] timing leaky_cswap4  mean ticks     24.9 vs     56.2  |t| =    65.25  -> leak (expected leak)

== Stage 6: EasyCrypt: extraction (normal and CT models) and proofs
   started 23:40:07
  [PASS] EasyCrypt type-checks Isa.ec (normal model)
  [PASS] EasyCrypt type-checks Isa_CT.ec (constant-time leakage model)
  [PASS] EasyCrypt type-checks Stackzero.ec (normal model)
  [PASS] EasyCrypt type-checks Stackzero_CT.ec (constant-time leakage model)
  [PASS] EasyCrypt type-checks Chacha20.ec (normal model)
  [PASS] EasyCrypt type-checks Chacha20_CT.ec (constant-time leakage model)
  [PASS] EasyCrypt type-checks Poly1305.ec (normal model)
  [PASS] EasyCrypt type-checks Poly1305_CT.ec (constant-time leakage model)
  [PASS] EasyCrypt type-checks Ct.ec (normal model)
  [PASS] EasyCrypt type-checks Ct_CT.ec (constant-time leakage model)
  [PASS] proofs/isa_correct.ec: add128 and SUBS carry flag correct
  [PASS] proofs/ct_proofs.ec: 5 constant-time (non-interference) proofs
  [PASS] fail_leaky_cswap4.ec: CT proof of a leaky function is rejected
```

```
  [PASS] fail_leaky_lookup16.ec: CT proof of a leaky function is rejected
  [PASS] fail_leaky_memeq.ec: CT proof of a leaky function is rejected

== Stage 7: performance (informational)
   started 23:42:51
  [INFO] kernel                                    ns/byte   cycles/B   (C: gcc -O2, 2400 MHz)
  [INFO] ChaCha20 block  Jasmin (scalar)             2.857       6.86
  [INFO] ChaCha20 block  C (scalar)                  2.504       6.01
  [INFO] ChaCha20 64 B   OpenSSL                     2.636       6.33
  [INFO] ChaCha20 4 KiB  OpenSSL (NEON)              1.001       2.40
  [INFO] Poly1305 4 KiB  Jasmin (radix 2^26)         2.271       5.45
  [INFO] Poly1305 4 KiB  C (radix 2^26)              0.905       2.17
  [INFO] Poly1305 4 KiB  OpenSSL                     0.504       1.21

== Stage 8: expected compilation failures (ARMv8-A back-end limitations)
   started 23:42:56
  [PASS] adc_imm           rejected: ADC has no immediate operand form.
  [PASS] add32_imm         rejected: 32-bit ADD accepts only imm12 (64-bit ADD materialises large constants automatically).
     ↪
  [PASS] args9             rejected: Export functions take at most 8 arguments (no stack-passed arguments).
  [PASS] csel_imm          rejected: Conditional moves take register operands only.
  [PASS] logic_imm         rejected: Logical operations accept only bitmask immediates.
  [PASS] mul_imm           rejected: MUL has no immediate operand form.
  [PASS] mulu32            rejected [internal compiler error]: 32x32->64 multiplication into two registers (mulu_32):
     ↪ internal compiler error.
  [PASS] mulu64            rejected [internal compiler error]: 64x64->128 multiplication (mulu) is not implemented:
     ↪ internal compiler error (x86-64 compiles it).
  [PASS] rotl_var          rejected: Rotate left by a register is not available (rotate right by -n instead).
  [PASS] sbb               rejected: Generic subtract-with-borrow: the borrow flag cannot be allocated (use SUBS + MVN +
     ↪ ADC).
  [PASS] shift_unmasked    rejected: Variable shifts must be written x << (n & 63).
  [PASS] slh               rejected: Selective Speculative Load Hardening (init_msf/update_msf/protect) is not implemented
     ↪  on ARMv8-A, although jasmin-ct --sct accepts the program.
  [PASS] stackzero_loopsct rejected: Stack zeroization strategy loopSCT is not supported on ARMv8-A.
  [PASS] swap32            rejected [internal compiler error]: #swap on 32-bit registers: internal compiler error (64-bit
     ↪ swap works).
  [PASS] u8_arith          rejected: There are no 8/16-bit general-purpose registers on AArch64.
  [PASS] umull             rejected: UMULL / UMADDL exist in the EasyCrypt model (JModel_armv8a.ec) but are not Jasmin
     ↪ intrinsics.
  [PASS] vector            rejected: No vector (NEON/SIMD) register bank on this target.
  negative tests: 17 passed, 0 failed

== Summary: 161 passed, 0 failed
./run-all.sh  388.06s user 9.37s system 94% cpu 7:01.36 total
```